ECO-TI

Entrega do aluno / Unidade VI

Protótipo acadêmico de central de chamados de TI. A entrega inclui abertura manual e assistida, organização de filas, encaminhamentos, classificação sugerida, comentários e histórico. A aplicação é somente front-end, responsiva e usa dados fictícios.

1. Identificação e estado das entregas

#	Entrega	Estado observado
1	Vercel	Impedimento documentado: Login da Vercel ainda não comprovado; página de autenticação aberta para o usuário.
2	GitHub e hash	Enviado ao repositório público. URL e hash completo abaixo.
3	Testes	28 unitários + 10 Playwright aprovados; sintaxe e build aprovados. CI: Concluída com sucesso no GitHub Actions.
4	Evidências visuais	Dois PNGs ECO-TI; vídeo WebM e trace ZIP da conversa assistida.
5	Autonomia	Matriz com cinco cenários, decisão justificada, controle e reversão.
6	Relatório	Documento com execução, autorizações, limites e reversão; páginas contadas após a geração.

Repositório público: https://github.com/bruno-df/ECO-TI

Hash da versão da aplicação e dos testes:

`2e6a042aa8cb8bbb31663bddab21d61c4b04d1aa`

O hash identifica a implementação verificada. A documentação final pode estar em um commit posterior; isso evita inserir no relatório um hash que dependeria do próprio arquivo.

Ferramenta efetiva: Codex. O enunciado usa o nome Claude; não houve execução do Claude nesta sessão. O registro de autoria e execução utiliza a ferramenta realmente empregada.

Projeto independente do SIGD e sem vínculo, integração ou autorização institucional da SEMA. A escolha da aplicação é didática.

2. Execução e testes

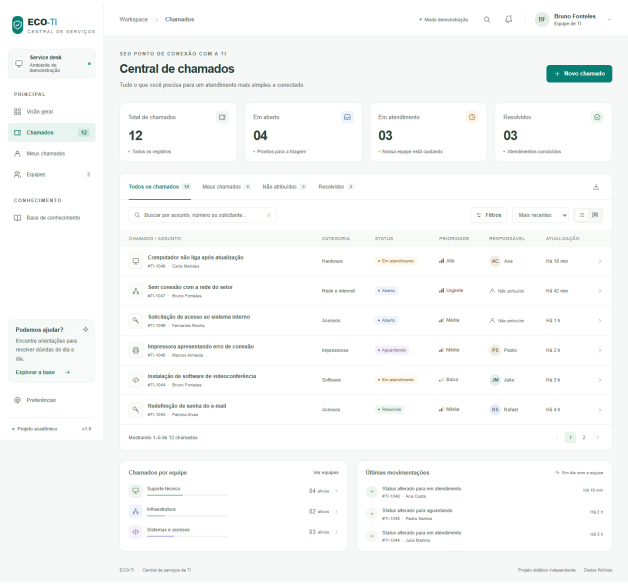
O agente implementou o front-end e o assistente por regras locais, renomeou a aplicação para ECO-TI, preparou o versionamento e a CI, executou verificações e produziu as evidências. O assistente não utiliza um modelo de linguagem, chave ou serviço externo de IA.

Verificação	Resultado verificável
Sintaxe e build	JavaScript aprovado; arquivos estáticos em dist/. Node v24.22.0, win32/x64.
Regras locais	28 testes aprovados, sem falhas; log TAP preservado.
Interface	Playwright 1.64.1 / Chromium: 10 aprovados, 0 falhas, 0 instáveis. Desktop 1440 x 1000 e mobile 390 x 844.
CI no GitHub	Concluída com sucesso no GitHub Actions. https://github.com/bruno-df/ECO-TI/actions/runs/38943481645

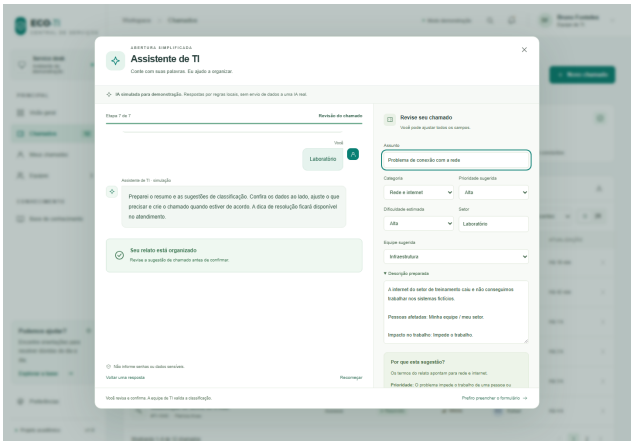
Os cenários verificam busca, filtros e quadro; criação manual, atribuição e persistência; conversa assistida, revisão de prioridade e dicas; relato vago e migração ao formulário; visão do solicitante e menu móvel.

A primeira execução E2E teve quatro falhas nos seletores dos testes. Após usar o papel acessível combobox, os dez testes passaram. Os testes simulam a revisão do solicitante; não representam cliques de aprovação feitos pelo humano.

Duas capturas com chamados fictícios



Print 1 - Central e chamados fictícios.



Print 2 - Assistente e revisão da sugestão.

Os PNGs originais estão em entrega/prints/. Vídeo e trace estão em entrega/playwright/. Logs, JSONs e resumos ficam em entrega/resultados/. A emulação móvel foi feita em Chromium; não comprova testes em Safari/iOS ou aparelho físico.

3. Autonomia e aprovação humana

#	Cenário	Decisão	Justificativa	Controle/reversão
1	Editar o front-end e a marca	Executar no escopo	Pedido explícito; mudanças locais, verificáveis e versionáveis.	Diff, testes e git revert.
2	Classificar e sugerir resolução	Sugerir e pedir revisão na interface	A simulação pode errar; solicitante confirma a abertura e a equipe valida.	Campos editáveis e histórico preservado.
3	Testes e capturas fictícias	Executar autonomamente	Evidências solicitadas, em contextos novos sem contas reais.	Logs, trace e suíte reproduzível.
4	Publicar no GitHub/Vercel	Executar após autenticação e validação	Destino público indicado pelo usuário. A autorização não abrange outras contas ou repositórios.	Verificar remoto/URL; sem force push; novo deploy ou revert.
5	Segredos, dados reais ou ações destrutivas fora do escopo	Interromper e obter autorização específica	Pode ampliar acesso ou causar perda irreversível; não faz parte da demonstração.	Não executado; segredos fora do Git e exportação antes de restaurar.

O que o humano efetivamente aprovou

- Construção do sistema de chamados somente front-end e responsivo, independente do SIGD.

- Visões de solicitante e equipe de TI, com troca de visão.

- Demonstração de assistente, classificação sugerida e dica de resolução, sem exigir integração completa com IA.

- Novo nome ECO-TI, repositório público bruno-df/ECO-TI e as seis entregas da atividade.

- Intenção de concluir a autenticação do Git e o login da Vercel. Intenção não é comprovação de login ou publicação.

Aprovação de escopo não equivale a aprovação humana de cada valor gerado nos testes. Os commits do agente usam identidade técnica fictícia do projeto, sem simular autoria humana.

4. Limites, publicação e reversão

Limites da aplicação

- Não há backend, autenticação ou autorização real. A troca de perfil é apenas uma visão da interface.

- Os dados ficam no navegador e origem atuais; não são compartilhados entre pessoas ou dispositivos. Limpar o armazenamento pode apagá-los.

- A IA é simulada por regras locais. Dificuldade e prioridade são estimativas; dicas não são diagnóstico nem executam alterações em equipamentos.

- Anexos guardam somente nome e tamanho. Não são enviados a uma equipe ou serviço externo.

Estado de publicação observado

GitHub: Enviado ao repositório público. Envio à branch main concluído e hash confirmado pela API pública do GitHub.

Vercel: Impedimento documentado: Login da Vercel ainda não comprovado; página de autenticação aberta para o usuário.

CI: Concluída com sucesso no GitHub Actions. https://github.com/bruno-df/ECO-TI/actions/runs/38943481645

Como reverter

- Código público: criar um novo commit com git revert HASH_DA_ALTERACAO e enviar ao remoto; não substituir o histórico com force push.

- Vercel: republicar uma versão anterior verificada ou publicar o commit de reversão; confirmar a URL depois.

- Dados locais: exportar JSON em Preferências antes de restaurar. A restauração remove registros do navegador após confirmação. A chave histórica foi preservada para compatibilidade.

Reproduzir a validação

```
npm ci
npx playwright install chromium
npm run check
npm test
npm run test:e2e
npm run build
```

Fontes e arquivos de referência

Manifesto, matriz, registros e instruções detalhadas acompanham esta entrega. Fontes técnicas oficiais: playwright.dev/docs/ci-intro e vercel.com/docs/git/vercel-for-github.